Đại học Bách khoa Hà Nội

Trường Công nghệ thông tin và truyền thông

----- ꕥ 🕮 ꕥ -----

Project 1

Báo cáo tổng kết

Xây dựng chương trình điều khiển vườn thông minh

Người thực hiện: Vũ Tiến Chiến – 20235279

Mã học phần: IT3150

Người hướng dẫn: Nguyễn Minh Huyền

Mã lớp: 755566

Hà Nội, tháng 1 – 2026

I. Giới thiệu đề tài

- Hiện nay, khi ngành công nghiệp bán dẫn phát triển mạnh, nhu cầu về những thiết bị thông minh ngày càng trở lên phổ biến hơn. Trong đó, xây dựng các tính năng điều khiển thiết bị từ xa luôn là công việc đầu tiên được quan tâm vì nó cho phép người dùng có thể tương tác với thiết bị từ các khoảng cách khác nhau và có ứng dụng rộng rãi trong nhiều lĩnh vực. Ví dụ:

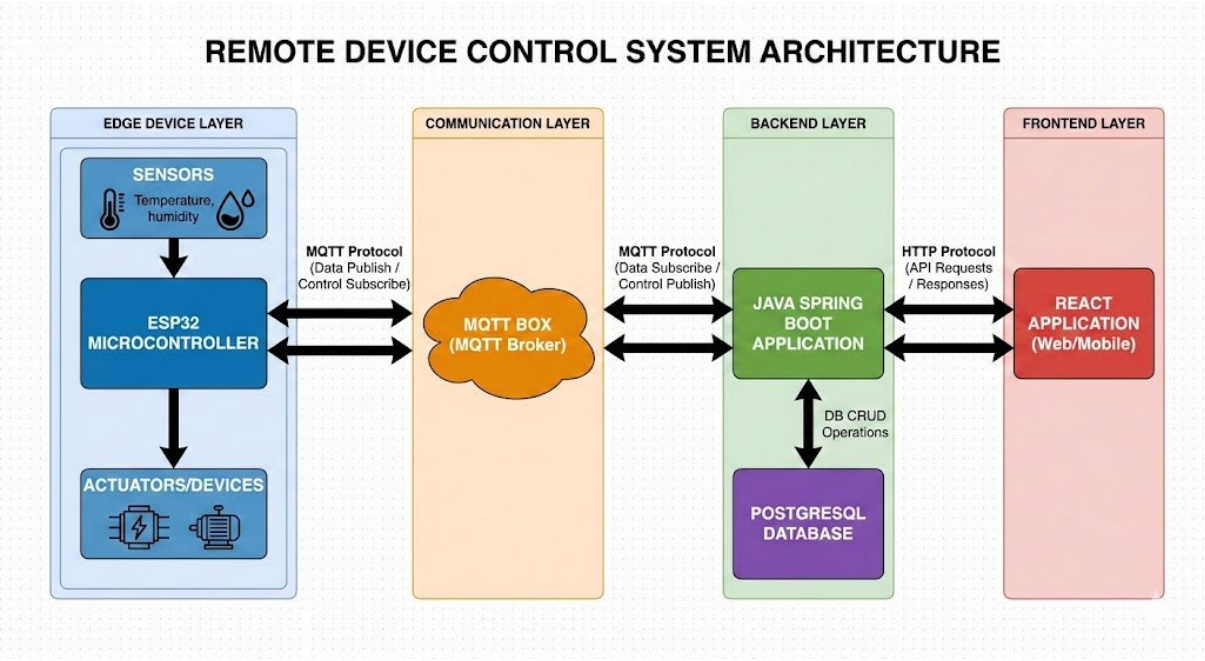
- Điều khiển các thiết bị gia đình như bình nóng lạnh, điều hòa, máy hút ẩm,… mà không cần phải ở nhà.
- Điều khiển các thiết bị như drone, xe mini,… thám hiểm, quan sát những nơi con người khó tự khám phá.
- Tự động hóa nhiều thiết bị mà không cần con người can thiệp.

- Trong đề tài lần này, chương trình sẽ tập trung vào các tính năng hỗ trợ phát triển một hệ thống vườn thông minh gồm:

- Lấy dữ liệu cảm biến (nhiệt độ, độ ẩm, ánh sáng, độ ẩm đất) để hiển thị cho người dùng theo thời gian thực, đồng thời hiển thị thời gian cập nhật cuối.
- Quản lý, thêm, xóa các thiết bị.
- Điều khiển thiết bị từ xa bằng cách nhấn nút.
- Lên lịch tự hoạt động (bật và tắt) cho thiết bị.
- Theo dõi hoạt động của thiết bị (thời gian bật và tắt).

II. Kiến trúc hệ thống

1. Kiến trúc tổng thể



Hình 1: Kiến trúc tổng thể của hệ thống.

Frontend	Framework ReactJS dùng Typescript, Tailwind CSS
Backend	Java Spring Boot

Database	PostgreSQL
Hardware programming	PlatformIO
Containerization	Docker
Deployment	Docker Compose

a. Frontend: ReactJS dùng Typescript, Tailwind CSS

- ReactJS: là một thư viện JavaScript mã nguồn mở được phát triển bởi Facebook, chuyên dùng để xây dựng giao diện người dùng (User Interface) cho các ứng dụng web hiện đại. ReactJS hoạt động dựa trên mô hình **component-based**, cho phép chia giao diện thành các thành phần độc lập, có khả năng tái sử dụng cao.

- Phù hợp để xây dựng giao diện động, cập nhật dữ liệu theo thời gian thực (real-time), rất cần thiết cho các hệ thống giám sát và điều khiển vườn thông minh.
- Cấu trúc component giúp chia nhỏ giao diện (dashboard, điều khiển tưới nước, bật/tắt đèn…) một cách rõ ràng.

- TypeScript: là một ngôn ngữ lập trình được phát triển bởi Microsoft, mở rộng từ JavaScript bằng cách bổ sung hệ thống kiểu tĩnh (static typing). TypeScript giúp phát hiện lỗi sớm trong quá trình phát triển, tăng tính ổn định và dễ bảo trì cho ứng dụng.

- Tăng độ an toàn kiểu dữ liệu, giảm lỗi trong quá trình phát triển.
- Phù hợp với các dự án có cấu trúc rõ ràng và cần bảo trì lâu dài.

- Tailwind CSS: là một framework CSS theo hướng utility-first, cung cấp sẵn các lớp CSS nhỏ gọn để xây dựng giao diện nhanh chóng. Thay vì viết CSS thủ công, Tailwind cho phép thiết kế giao diện trực tiếp trong file HTML hoặc JSX.

- Giúp xây dựng giao diện nhanh chóng, đồng nhất.
- Giảm thời gian viết CSS thủ công, dễ chỉnh sửa và mở rộng giao diện.

=> Đóng vai trò là giao diện tương tác trực tiếp với người dùng.

b. Backend: Java Spring Boot

- Spring Boot là một framework thuộc hệ sinh thái Spring, được thiết kế nhằm đơn giản hóa việc phát triển các ứng dụng Java phía server. Spring Boot hỗ trợ sẵn các cấu hình mặc định, giúp lập trình viên nhanh chóng xây dựng các RESTful API mà không cần cấu hình phức tạp.

- Spring Boot hỗ trợ:

- Xây dựng API nhanh chóng.
- Cấu hình đơn giản, dễ triển khai.

- Java là ngôn ngữ:

- Ổn định, phổ biến.
- Phù hợp cho các hệ thống xử lý nghiệp vụ và kết nối cơ sở dữ liệu.

- Hệ sinh thái Spring hỗ trợ tốt cho:

- Kết nối MQTT.

- Bảo mật.
- Xử lý song song và mở rộng hệ thống.

=> Đóng vai trò là trung tâm xử lý logic nghiệp vụ.

c. Database: PostgreSQL

- PostgreSQL là một hệ quản trị cơ sở dữ liệu quan hệ mã nguồn mở, nổi tiếng với độ tin cậy cao và khả năng xử lý dữ liệu phức tạp. PostgreSQL hỗ trợ đầy đủ các chuẩn SQL và nhiều tính năng nâng cao như transaction, indexing và constraint.

- PostgreSQL có:

- Độ tin cậy cao.
- Khả năng xử lý dữ liệu lớn và phức tạp.

- Hỗ trợ tốt:

- Các truy vấn thống kê.
- Quan hệ dữ liệu rõ ràng (thiết bị, cảm biến, lịch sử đo đạc).

=> Phù hợp cho các hệ thống lưu trữ dữ liệu cảm biến theo thời gian.

d. Hardware programming: PlatformIO

- PlatformIO là một nền tảng phát triển phần mềm dành cho các hệ thống nhúng và IoT. PlatformIO cung cấp môi trường lập trình thống nhất cho nhiều loại vi điều khiển khác nhau, trong đó có ESP32.

- Hỗ trợ quản lý thư viện tốt.

- Dễ tích hợp với nhiều loại vi điều khiển.

- Phù hợp cho phát triển và mở rộng dự án.

e. Containerization và Deployment: Docker & Docker Compose

- Docker là một nền tảng mã nguồn mở cho phép đóng gói ứng dụng và các thành phần phụ thuộc vào các container độc lập. Các container này có thể chạy nhất quán trên nhiều môi trường khác nhau mà không phụ thuộc vào hệ điều hành nền.

- Docker Compose cho phép:

- Quản lý nhiều container cùng lúc.
- Cấu hình kết nối giữa backend và database một cách đơn giản.
- Phù hợp với các dự án nhỏ và vừa như đồ án này.

f. Giao thức MQTT

- MQTT (Message Queuing Telemetry Transport) là một giao thức truyền thông nhẹ, được thiết kế cho các hệ thống IoT và các thiết bị có tài nguyên hạn chế. MQTT sử dụng mô hình publish/subscribe, cho phép các thiết bị trao đổi dữ liệu thông qua một broker trung gian.

- Lý do sử dụng MQTT cho giao tiếp thiết bị – server:

- Tiêu thụ băng thông thấp.

- Phù hợp với thiết bị tài nguyên hạn chế như ESP32.
- Hỗ trợ mô hình publish/subscribe.
- Dễ mở rộng khi số lượng thiết bị tăng.

e. Giao thức HTTP

- HTTP (Hypertext Transfer Protocol) là giao thức truyền thông phổ biến trên nền tảng web, cho phép trao đổi dữ liệu giữa client và server theo mô hình request/response.

- Lý do sử dụng HTTP:

- Phổ biến, dễ triển khai.
- Phù hợp với mô hình RESTful API.
- Dễ tích hợp với ReactJS.
- Đáp ứng tốt các yêu cầu CRUD (xem dữ liệu, điều khiển thiết bị).

2. Phần cứng

a. Vi xử lý: KIT WIFI ESP32-WROOM-32 (Type C)

Vi xử lý ESP32 đóng vai trò bộ não điều khiển các cảm biến và thiết bị. Nó có nhiệm vụ:

- Nhận dữ liệu cảm biến từ các cảm biến, đóng gói trong JSON rồi gửi lên MQTT Box mỗi 5s một lần.
- Nhận lệnh điều khiển dạng JSON được chương trình phần mềm gửi lên MQQT Box, phân giải và điều khiển thiết bị.
- Mỗi khi bật hoặc tắt thiết bị, nó sẽ ghi lại thời gian hoạt động rồi đóng gói trong JSON rồi gửi lên MQTT Box để chương trình lấy dữ liệu.

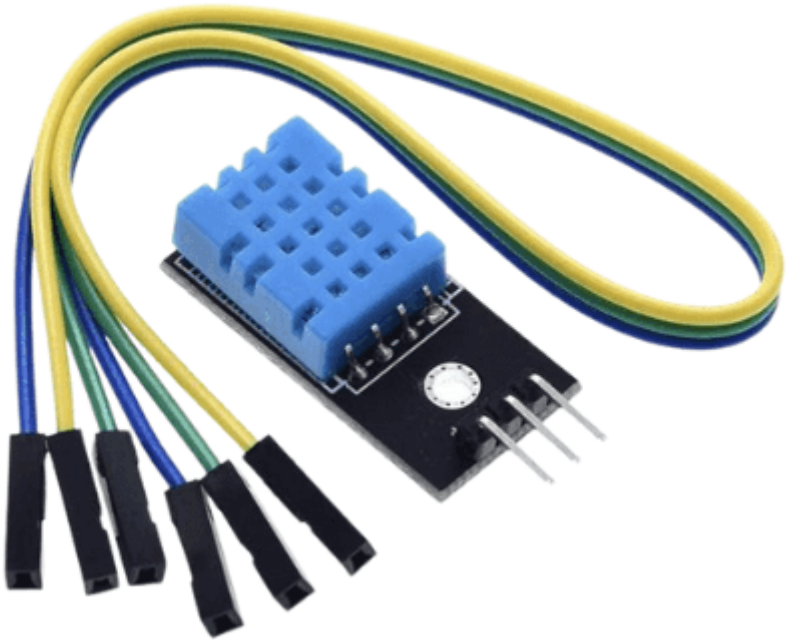
Hình 2: KIT WIFI ESP32-WROOM-32 (Type C).

IC chính	Wifi BLE SoC ESP32 ESP-WROOM-32
Điện áp sử dụng	2.2V~3.6VDC
Dòng điện sử dụng	~90mA
Nhân xử lý trung tâm	ESP32-D0WDQ6 Dual-core low power Xtensa® 32-bit LX6 microprocessors.
ROM	448KBytes
SRAM	520 KBytes
	8 KBytes SRAM in RTC SLOW
	8 KBytes SRAM in RTC FAST
WiFi	802.11 b/g/n/d/e/i/k/r (802.11n up to 150 Mbps)
Bluetooth	Bluetooth v4.2 BR/EDR and BLE specification
Wi-Fi mode	Station/softAP/SoftAP+station/P2P
Bảo mật	WPA/WPA2/WPA2-Enterprise/WPS
Encryption	AES/RSA/ECC/SHA
Giao thức mạng	IPv4, IPv6, SSL, TCP/UDP/HTTP/FTP/MQTT
Interfaces	SD-card, UART, SPI, SDIO, I2C, LED PWM, Motor PWM, I2S , IR, GPIO, capacitive touch sensor, ADC, DAC, Hall sensor, temperature sensor

Hình 3: Bảng thông số kỹ thuật của ESP32.

b. Các cảm biến

- Cảm biến nhiệt độ, độ ẩm: Module DHT11

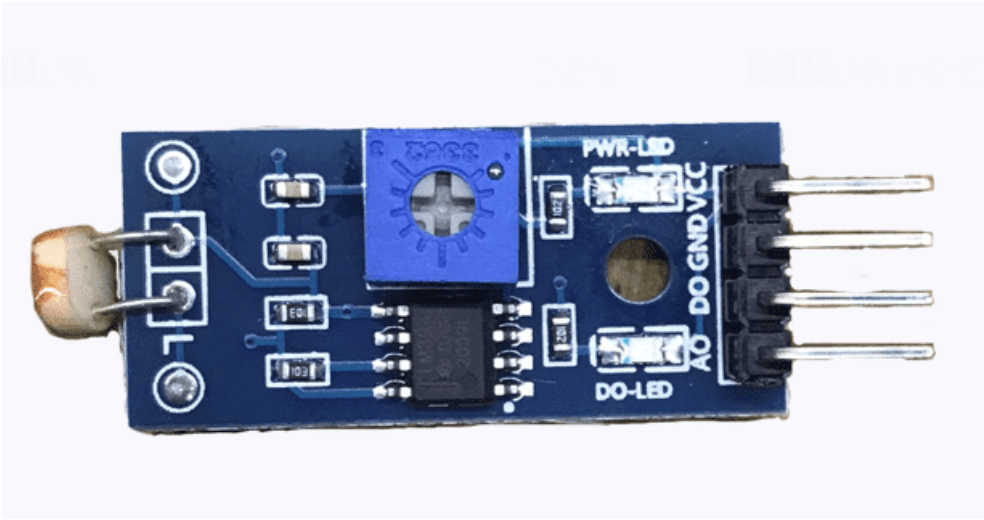


Hình 4: Module DHT11.

Điện áp hoạt động	3.3 đến 5V
Dải đo nhiệt độ	-20 đến 60*C
Sai số nhiệt độ	2*C
Dải đo độ ẩm	20% ~ 90%
Sai số độ ẩm	5%
Kích thước PCB	2.8x1.4cm
Trọng lượng	8g

Hình 5: Bảng thông số kỹ thuật của DHT11.

- Cảm biến ánh sáng: Module Cảm Biến Ánh Sáng MS-CDS05

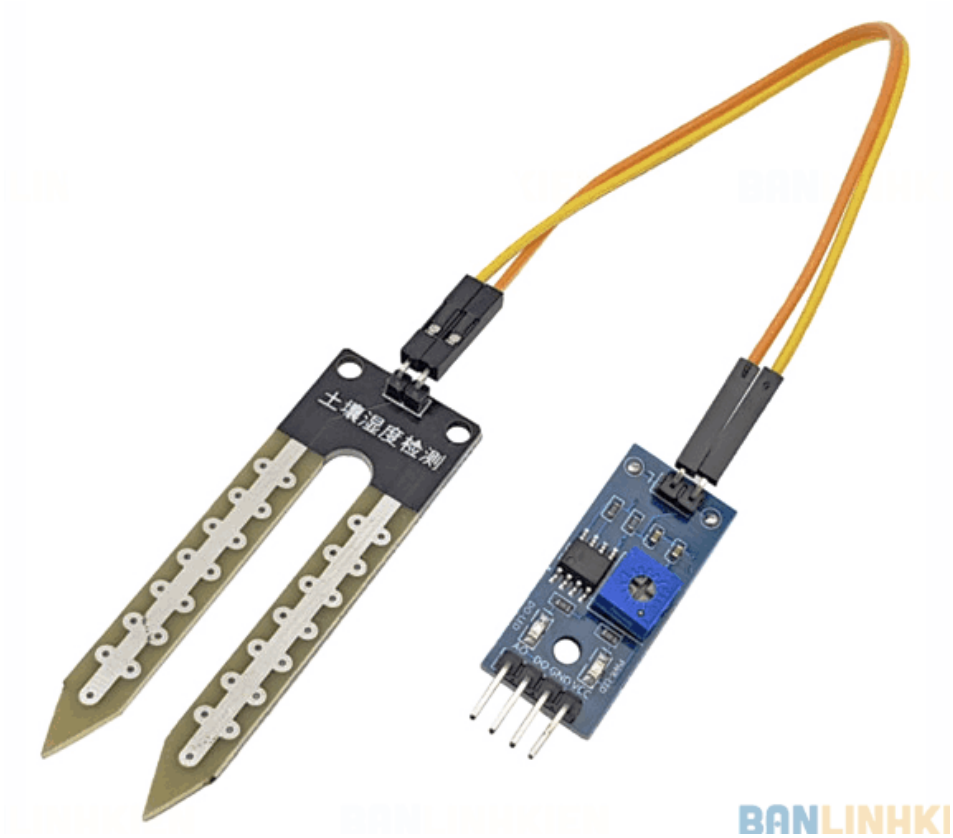


Hình 6: Module Cảm Biến Ánh Sáng MS-CDS05.

Loại cảm biến	cảm biến ánh sáng
Điện áp hoạt động	3.3-5 DVC
Đầu ra	AO: Tín hiệu analog
Điều chỉnh	DO: Tín hiệu 0 hoặc 1 (Sử dụng LM393)

Hình 7: Bảng thông số kỹ thuật của MS-CDS05.

- Cảm biến độ ẩm đất: Cảm Biến Đo Độ Ẩm Đất TH



Hình 8: Cảm Biến Đo Độ Ẩm Đất TH.

Điện áp sử dụng	3.3-5V
Đầu ra	AOUT, DOUT, VCC, GND
AOUT	Tín hiệu ADC
DOUT	Mức logic 0,1
IC so sánh	LM393
Kích thước PCB	3cm * 1.6cm

Hình 9: Bảng thông số kỹ thuật của TH.

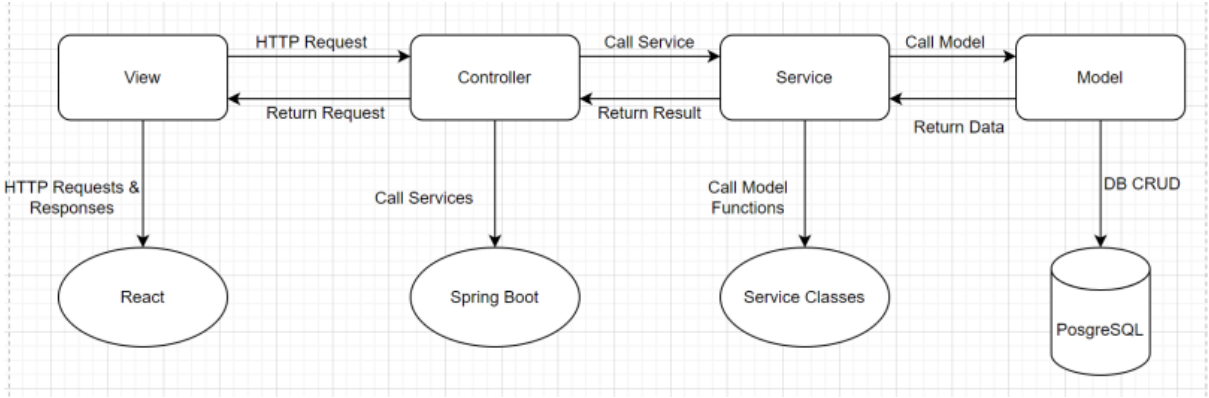
c. Thiết bị

Chúng ta có thể ứng dụng chương trình cho rất nhiều loại thiết bị khác nhau: đèn điện, quạt gió, máy hút mùi, máy bơm, bình nóng lạnh,… Lúc này ESP32 sẽ đóng vai trò thay thế công tắc, giúp chúng ta có thể bật và tắt thiết bị từ mọi nơi, mọi lúc cũng như có thể đặt lịch cho thiết bị tự động bật tắt từ trước. Trong đề tài này, để tiện lợi và tiết kiệm chi phí, chúng ta sẽ sử dụng đèn LED cùng với điện trở thay thế cho thiết bị thật.

Hình 10: Đèn LED và điện trở.

3. Phần mềm

- Chương trình phần mềm điều khiển thiết bị từ xa được xây dựng dựa trên khung của sơ đồ kiến trúc Model - View - Controller - Service (MVCS). Mẫu kiến trúc MVCS là phương pháp chia nhỏ các thành phần dữ liệu từ người dùng, dữ liệu trình bày thành các phần riêng biệt, tách biệt, dễ mở rộng và bảo trì, đồng thời tăng khả năng tái sử dụng mã.



Hình 11: Kiến trúc phần mềm của chương trình.

- Model: Là nơi định nghĩa các thực thể (Entity) của hệ thống: SensorData, Device, DeviceLog, Scheduler,… Sử dụng JPA (Hibernate) để ánh xạ các bảng trong PostgreSQL thành các lớp Java. Model bao gồm các thao tác CRUD cơ bản (qua Repository): tạo, sửa, xóa, truy vấn dữ liệu.

- Service: Chứa toàn bộ logic nghiệp vụ như: nhận dữ liệu từ vi xử lý, gửi điều khiển đến vi xử lý, ghi lại hoạt động của thiết bị,…; có thể gọi đến nhiều Repository hoặc các dịch vụ ngoài. Service được tách ra giúp Controller chỉ tập trung vào xử lý HTTP request mà không cần quan tâm tới xử lý logic nghiệp vụ.

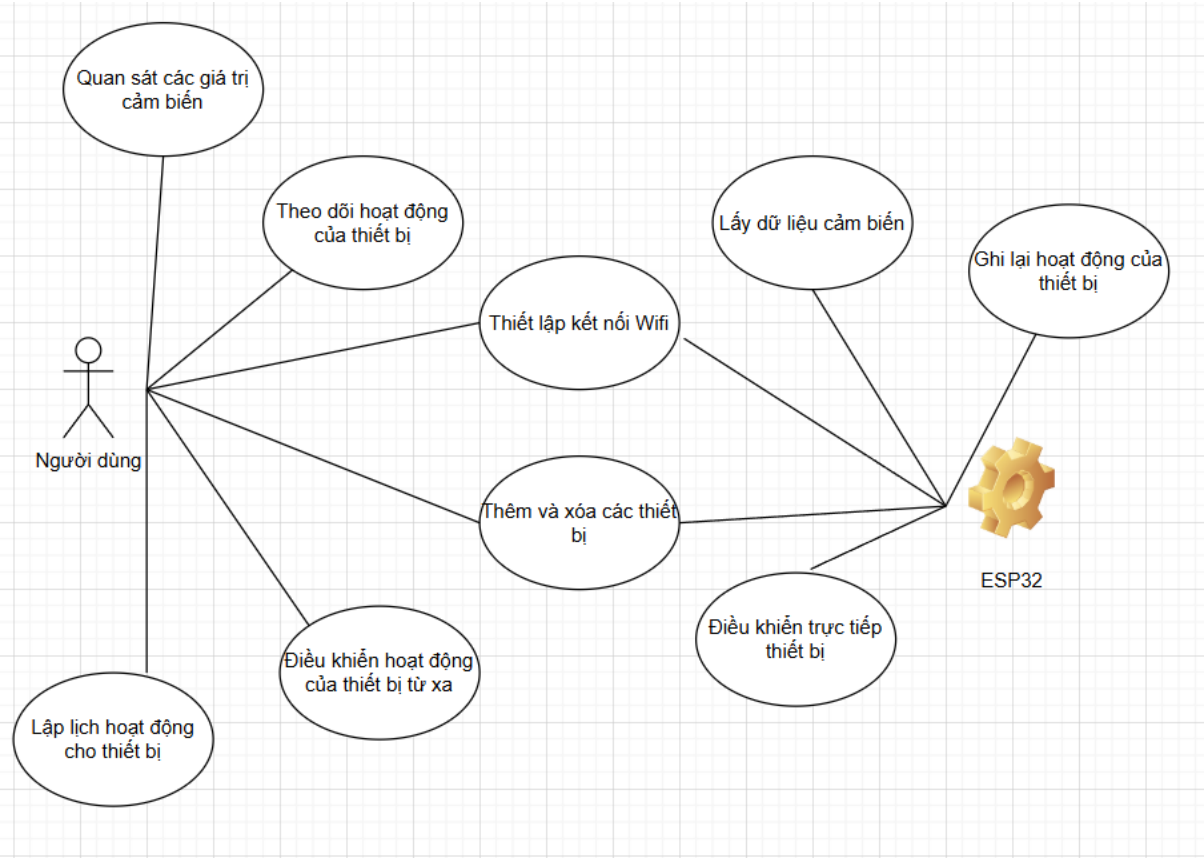
- Controller: Controller đóng vai trò tiếp nhận các HTTP Request từ frontend React, ủy quyền xử lý logic cho Service, sau đó trả kết quả (JSON) về frontend. Controller đóng vai trò trung gian, điều phối giữa View và Service.

- **View:** Giao diện người dùng được xây dựng bằng Framework ReactJS dùng Typescript, Tailwind CSS, đảm bảo việc hiển thị dữ liệu cho người dùng và tương tác với hệ thống, dễ sử dụng. Giao diện gồm các phần: hiển thị dữ liệu cảm biến, hiển thị các thiết bị hiện có và nút điều khiển, hiển thị màn hình lập lịch, hiển thị hoạt động của hệ thống.

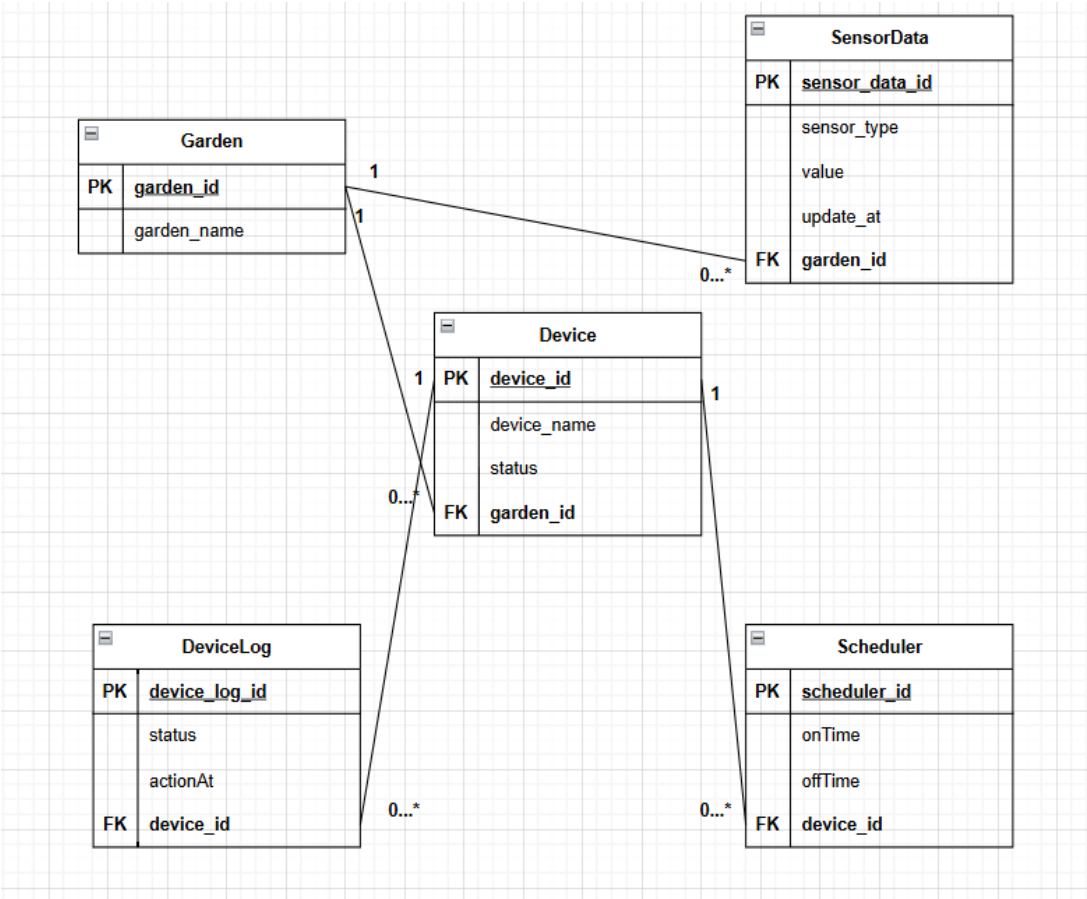
=> Việc sử dụng kiến trúc MVCS kết hợp với các công nghệ hiện đại như ReactJS (TypeScript), Spring Boot, PostgreSQL, JWT giúp hệ thống đạt được hiệu quả về hiệu năng xử lý, đảm bảo tính bảo mật, dễ mở rộng, dễ bảo trì, thân thiện với người dùng. Đây là một kiến trúc phù hợp với định hướng phát triển lâu dài và chuyên nghiệp, đồng thời bám sát kiến thức nền tảng của Công nghệ phần mềm.

III. Phân tích và thiết kế

1. Thiết kế hoạt động chung của hệ thống

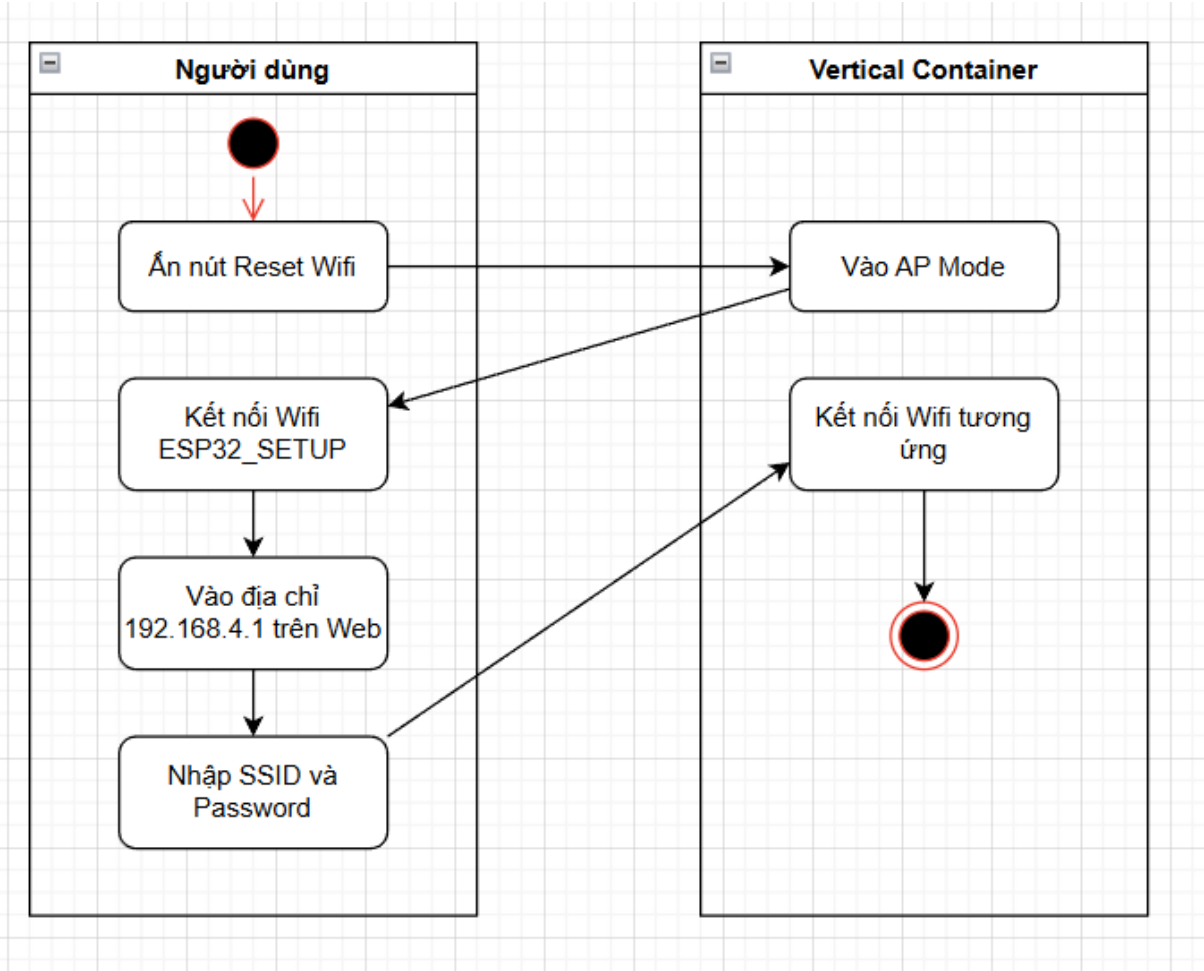


2. Thiết kế cơ sở dữ liệu

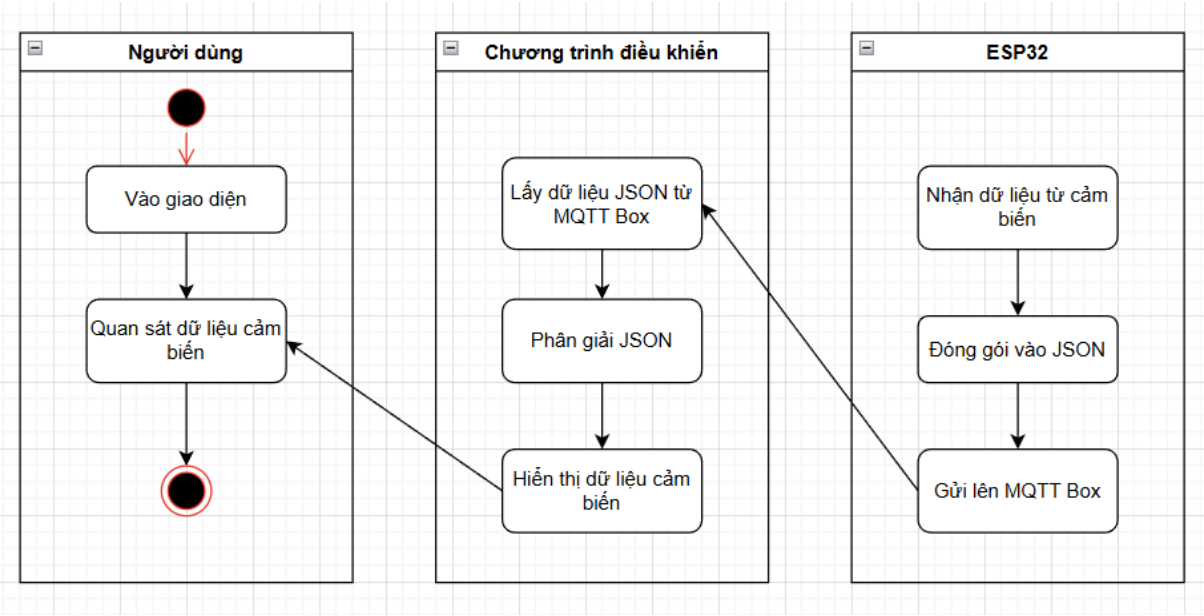


3. Thiết kế luồng hoạt động

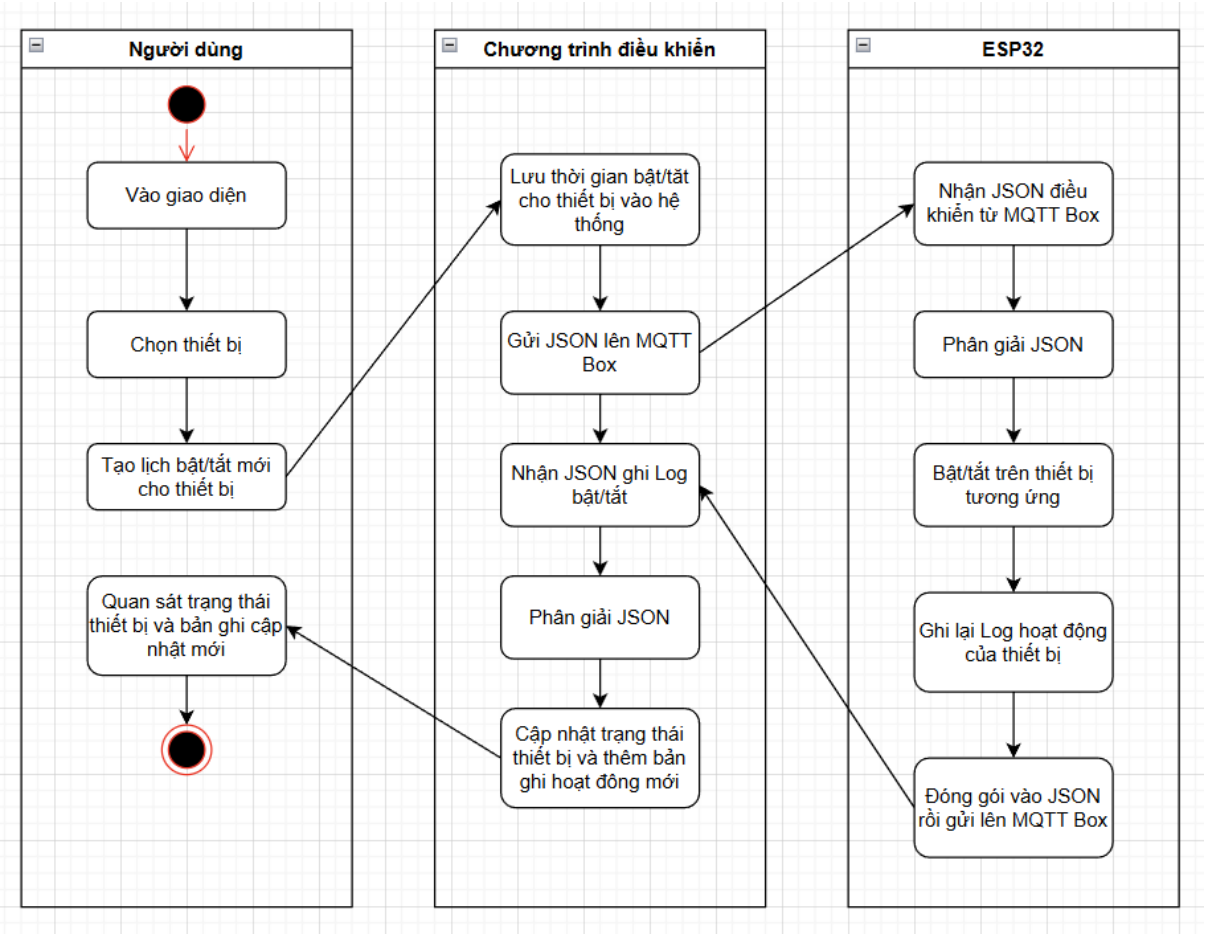
a. Thiết lập kết nối Wifi



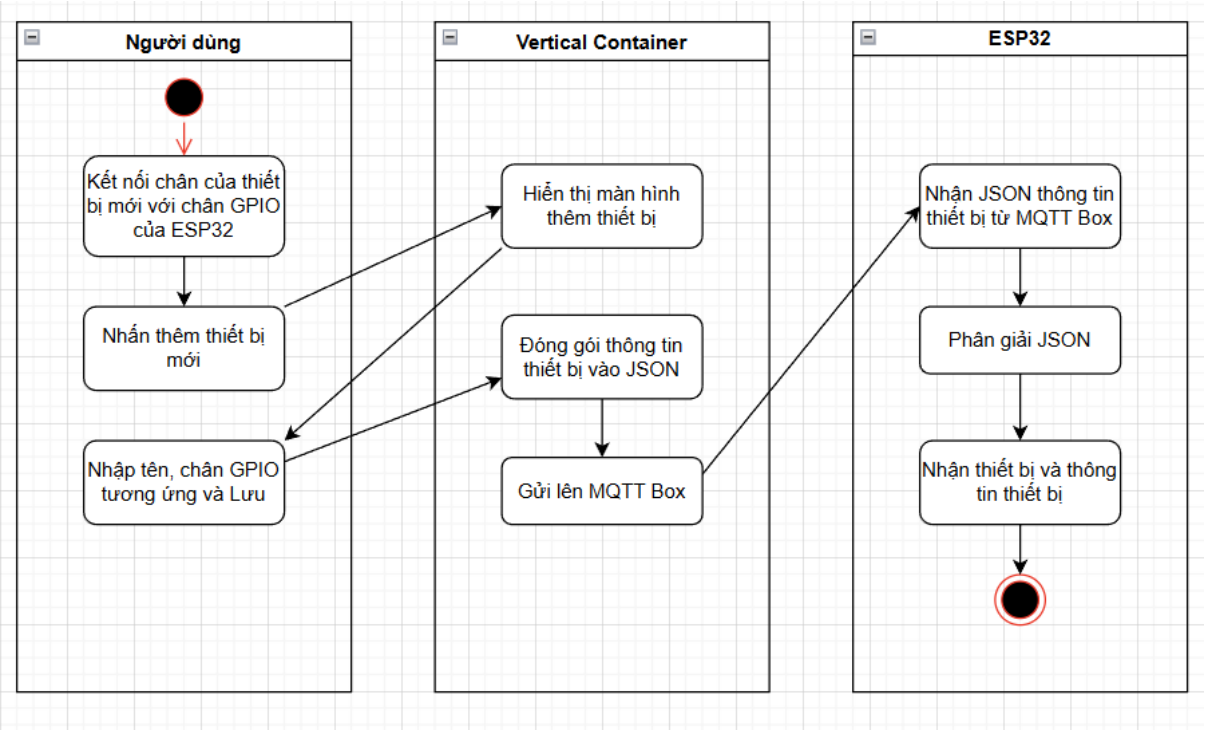
b. Quan sát các dữ liệu cảm biến



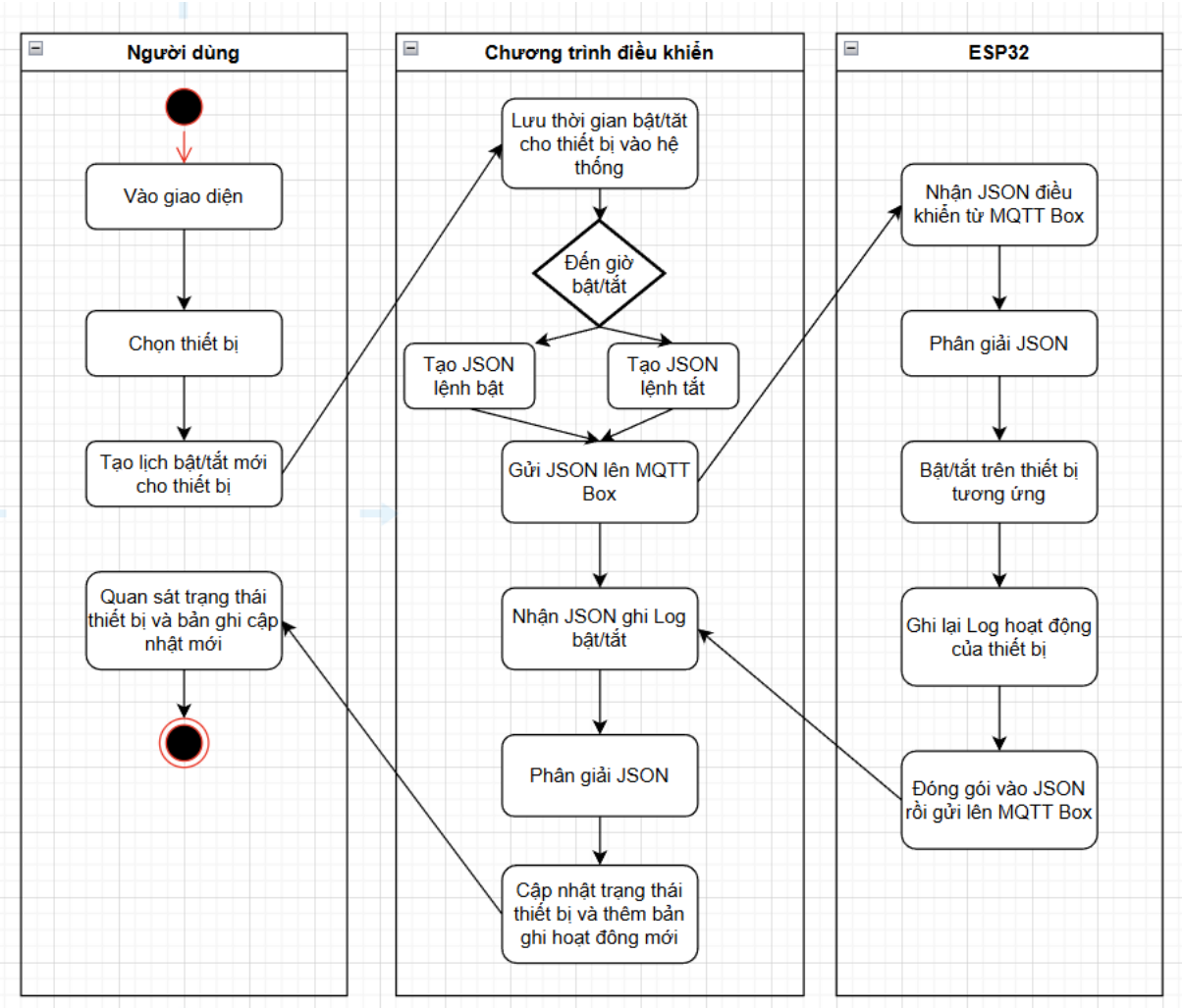
c. Điều khiển hoạt động của thiết bị



d. Thêm thiết bị mới

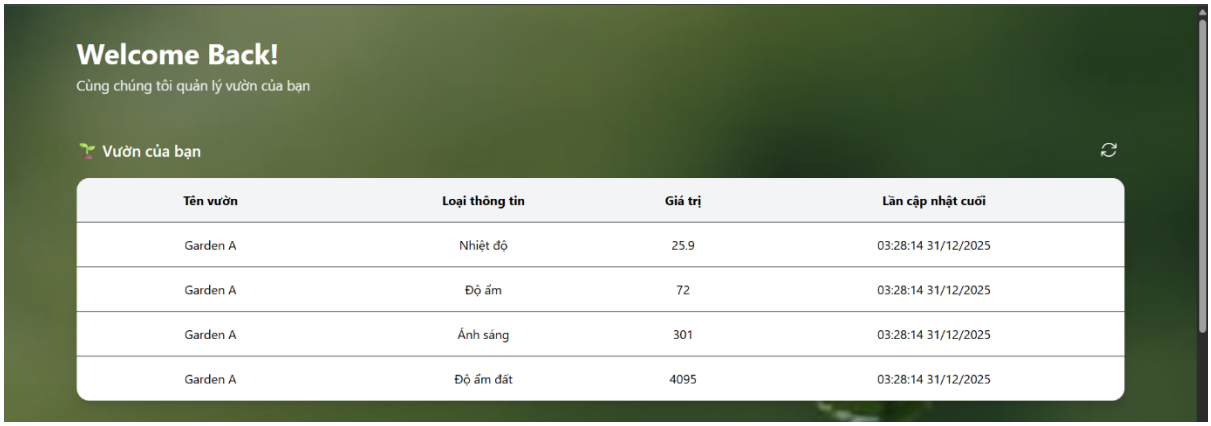


e. Lập lịch bật tắt tự động cho thiết bị

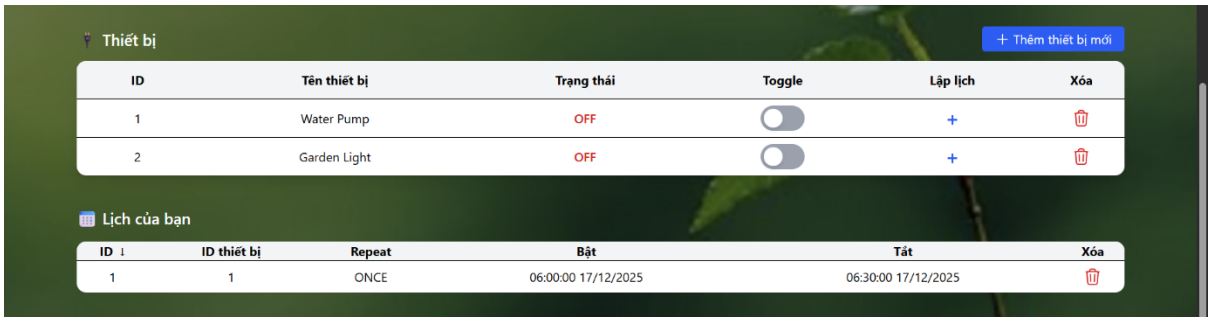


IV. Hoạt động thực tế của chương trình

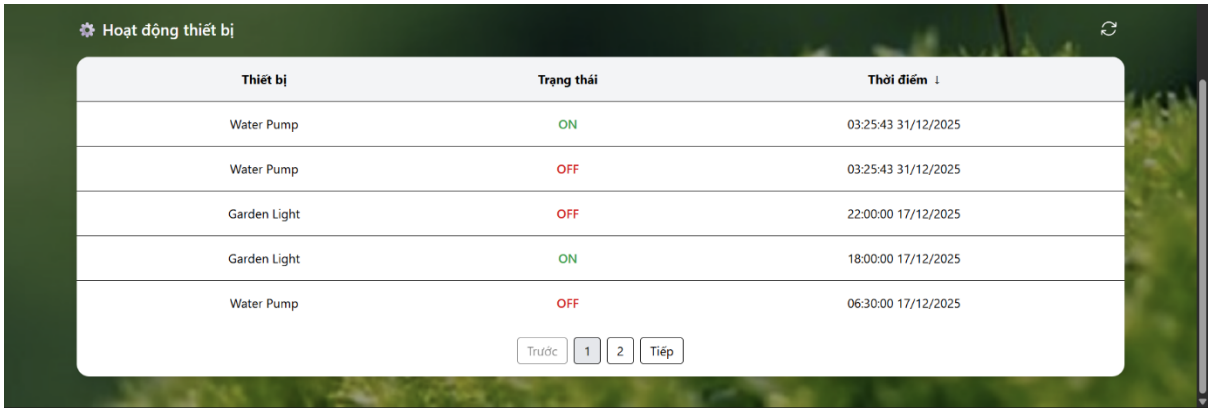
Hình 12: Nhập SSID và Password của Wifi cần kết nối trên địa chỉ 192.168.4.1.



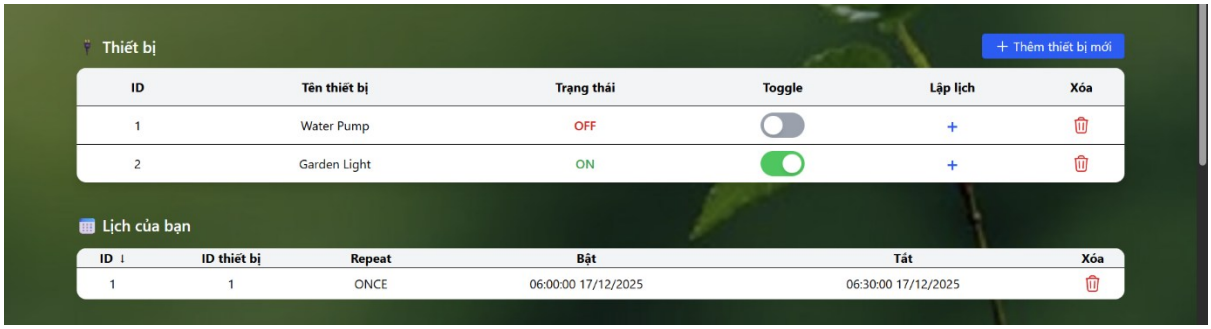
Hình 13: Giao diện để xem các dữ liệu cảm biến với thời gian cập nhật cuối tương ứng.



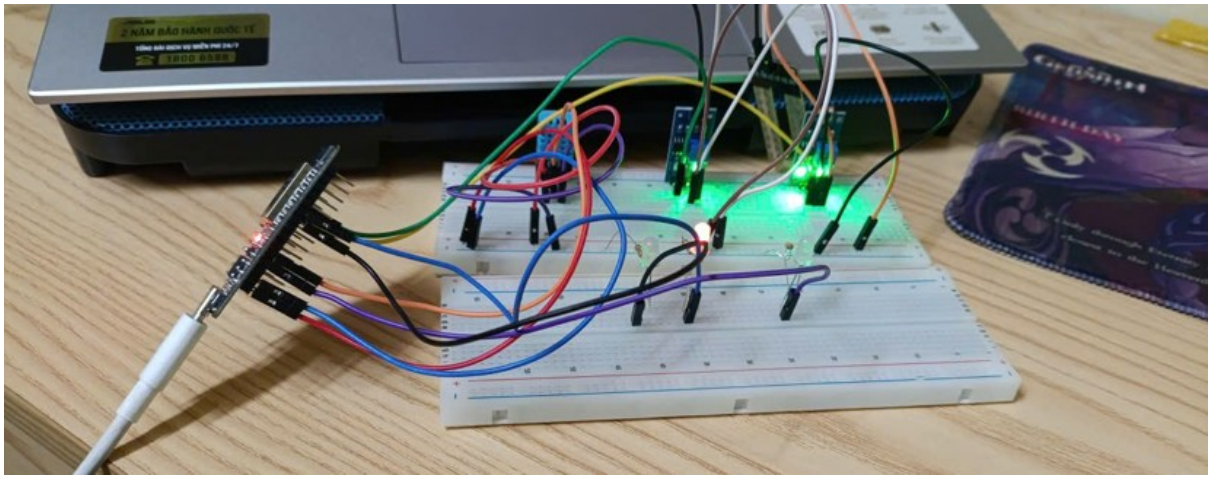
Hình 14: Giao diện xem thiết bị và lịch cùng với các nút thêm, xóa thiết bị hoặc lịch và nút bật/tắt thiết bị.



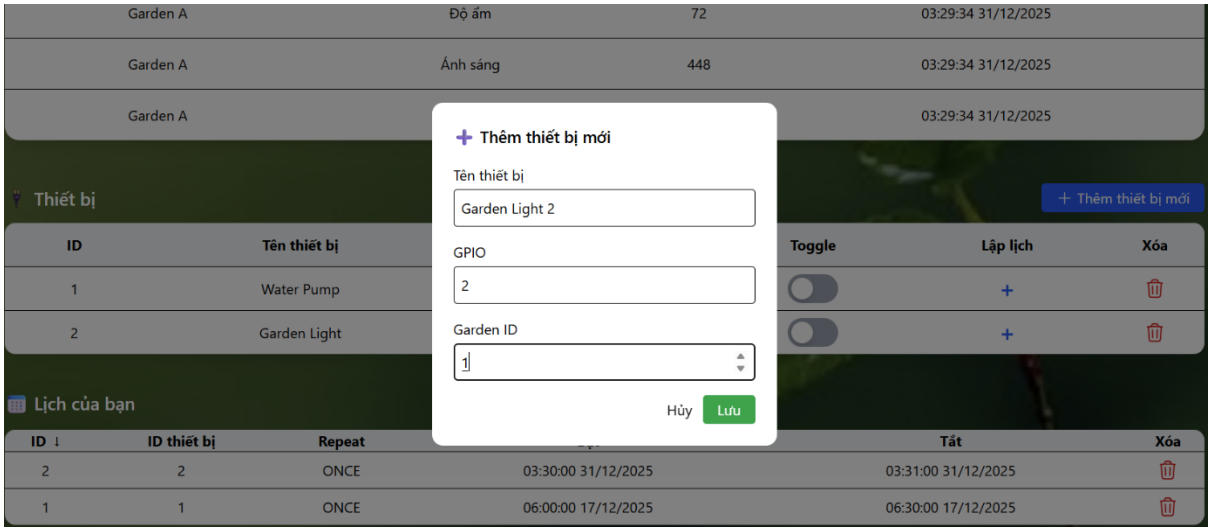
Hình 15: Giao diện theo dõi hoạt động của thiết bị.



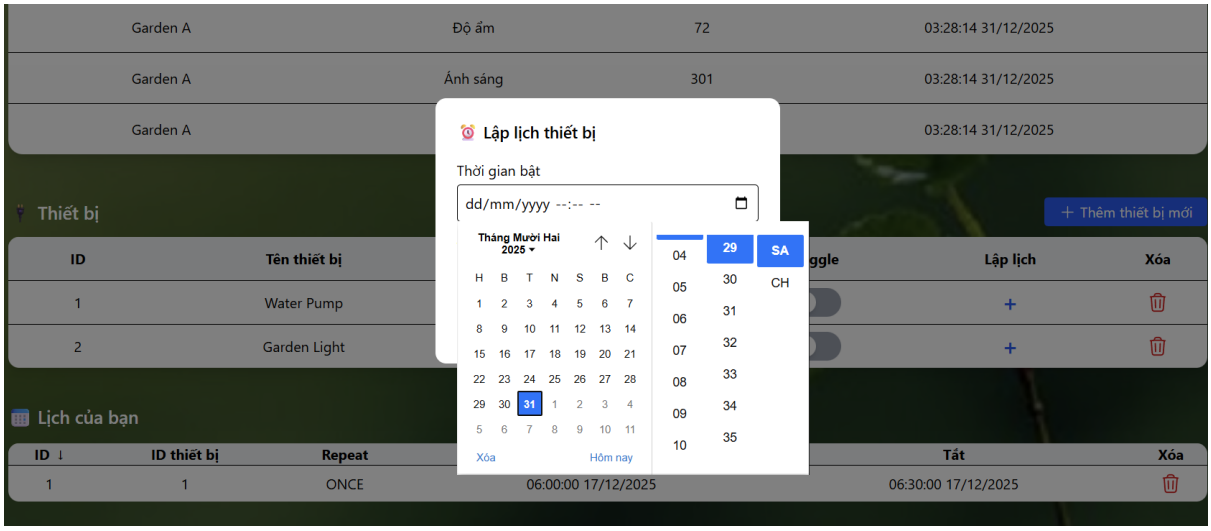
Hình 16: Khi ấn nút, thiết bị sẽ được bật/tắt và cập nhật trạng thái tương ứng.



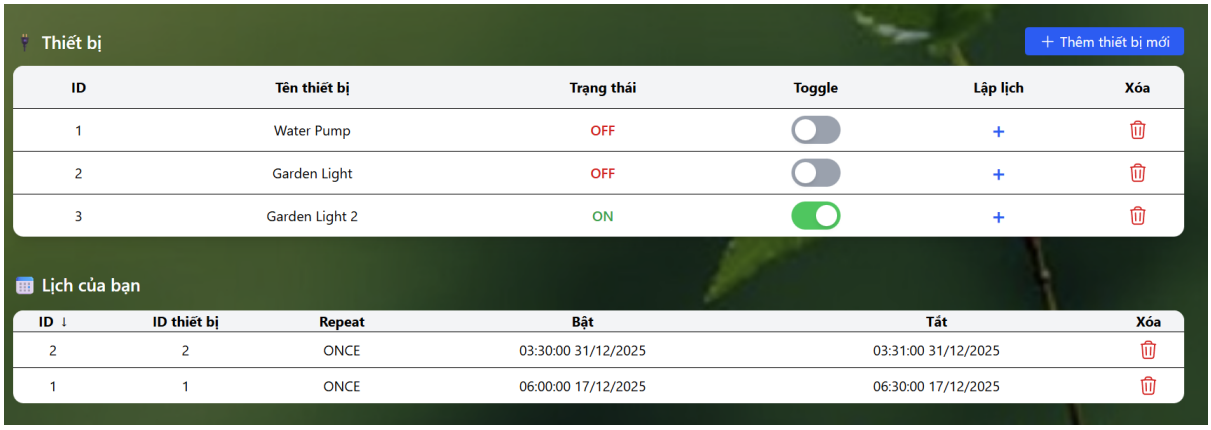
Hình 17: Đồng thời, trên thiết bị thật, thiết bị cũng sẽ bật/tắt tương ứng.



Hình 18: Giao diện để thêm thiết bị mới.



Hình 19: Màn hình thêm lịch hoạt động mới.



Hình 20:Lịch hoạt động mới và thiết bị mới được thêm vào, lúc này, có thể tương tác với thiết bị mới luôn.

V. Github Repository

https://github.com/Mymineme234/Smart-Garden.git